AEP — Agentes de Software para Projetos da Escola de TI

Entrega 1 — Especificação de Agentes Selecionados

Agente 1 — Agente de Testes de Software (QualityBridge) — agente próprio da equipe

Agente 2 — Agente Arquiteto — selecionado de outra equipe

Agente 3 — Agente de Documentação de Software — selecionado de outra equipe

AGENTE 1 — Agente de Testes de Software (QualityBridge)

Propósito e problema resolvido

O QualityBridge tem como objetivo automatizar a execução de testes já existentes em diferentes projetos de software. O agente centraliza a preparação, execução e análise dos testes, evitando tarefas repetitivas para a equipe. Ele também organiza os resultados e gera relatórios que facilitam a identificação de falhas. A proposta é permitir que o mesmo agente seja utilizado em projetos que utilizam diferentes ferramentas de testes.

Relação com a ODS escolhida

O projeto está relacionado à ODS 9 — Indústria, Inovação e Infraestrutura. Essa relação acontece de forma indireta, pois o agente contribui para a inovação tecnológica ao facilitar a verificação e manutenção de sistemas de software. A automação dos testes pode ajudar as equipes a encontrar falhas mais rapidamente, reduzir retrabalho e manter sistemas mais confiáveis.

Entradas, saídas e gatilho de acionamento

Entradas	Saídas	Gatilho de acionamento
Código do projeto, testes existentes, arquivo qualitybridge.yaml e dados necessários para os testes.	Relatórios em HTML e JSON, resultados em JUnit XML, logs e informações sobre os testes executados.	Sob demanda pela CLI no MVP. Futuramente, poderá ser acionado por eventos de CI ou de forma agendada.

Estilo arquitetural adotado

Será utilizada uma arquitetura em camadas, dividindo o sistema em partes responsáveis por diferentes funções, como entrada de comandos, execução dos testes, processamento dos resultados e geração dos relatórios. Essa organização facilita a manutenção do projeto e permite adicionar novas ferramentas de testes no futuro sem precisar alterar todo o sistema.

Padrões de projeto aplicados (GoF e GRASP)

Categoria	Padrão adotado	Justificativa breve
Criação (GoF)	Factory Method	Permite criar o executor adequado de acordo com a ferramenta de testes utilizada.
Estrutural (GoF)	Adapter	Permite utilizar diferentes ferramentas, como pytest e Playwright, seguindo uma mesma estrutura dentro do agente.
Comportamental (GoF)	Strategy	Permite escolher diferentes tipos de execução, como uma suíte completa ou uma suíte de testes mais rápida.
GRASP	Controller / baixo acoplamento	Um controlador será responsável por organizar a execução, mantendo as diferentes partes do sistema separadas.

Aplicação do padrão MVC

O projeto utilizará uma separação inspirada no padrão MVC, adaptada para uma aplicação de linha de comando.

- **Model:** contém as informações do projeto, dos testes e dos resultados.
- **View:** apresenta os resultados na CLI e nos relatórios gerados.
- **Controller:** recebe os comandos e coordena a execução dos testes.

Não haverá uma interface gráfica no MVP.

Diagrama de arquitetura do agente

```
Projeto / CLI
|
Configuração
|
QualityBridge
|
Seleção da suíte
|
Executor de testes
|
pytest / Playwright
|
Resultados
|
Relatórios (HTML / JSON / JUnit XML)
```

O agente recebe a configuração do projeto, seleciona os testes que serão executados, utiliza a ferramenta correspondente, coleta os resultados e gera os relatórios.

Base tecnológica de construção do agente

O QualityBridge será construído utilizando uma arquitetura própria, baseada em regras determinísticas, sem utilizar um modelo de linguagem de grande porte (LLM). Essa escolha permite que as decisões do agente sejam previsíveis e possam ser reproduzidas. O agente utilizará as configurações fornecidas pelo projeto e os resultados das ferramentas de testes para realizar seu trabalho. O agente não irá criar testes nem tentar descobrir regras de negócio automaticamente. Os testes e os resultados esperados serão fornecidos pelo próprio projeto.

Tecnologias e frameworks previstos

- Python 3: desenvolvimento do agente.
- Typer: criação da interface de linha de comando.
- Pydantic e PyYAML: leitura e validação das configurações.
- pytest: execução de testes em projetos Python.
- Playwright Test: execução de testes em aplicações web.
- Jinja2: geração dos relatórios HTML.
- JSON e SQLite: armazenamento dos resultados e histórico.
- Docker: criação de ambientes isolados para execução.
- GitHub Actions: integração contínua, prevista para uma etapa futura.

O MVP será testado em dois projetos diferentes, utilizando pytest e Playwright. Outras ferramentas poderão ser adicionadas posteriormente por meio de novos adaptadores.

Modelo de interação com os demais agentes

No MVP, o QualityBridge funcionará de forma independente, sem comunicação direta ou orquestração entre vários agentes. A ferramenta poderá ser executada localmente ou integrada a pipelines de automação. Futuramente, os relatórios em JSON poderão ser utilizados por outros agentes do ecossistema, como o Agente de Documentação. Essa integração futura não será necessária para que o QualityBridge execute os testes.

Referências utilizadas

NAÇÕES UNIDAS. Goal 9: Industry, innovation and infrastructure. Disponível em: https://sdgs.un.org/goals/goal9. Acesso em: 9 set. 2026.

FOWLER, Martin. Presentation Domain Data Layering. 2015. Disponível em: https://martinfowler.com/bliki/PresentationDomainDataLayering.html. Acesso em: 9 set. 2026.

GAMMA, Erich et al. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1995.

LARMAN, Craig. Applying UML and Patterns. 3. ed. Upper Saddle River: Prentice Hall, 2005.

PYTEST. Managing pytest's output. Disponível em: https://docs.pytest.org/en/7.1.x/how-to/output.html. Acesso em: 9 set. 2026.

MICROSOFT. Playwright: Command line. Disponível em: https://playwright.dev/docs/test-cli. Acesso em: 9 set. 2026.

AGENTE 2 — Agente Arquiteto

Propósito e problema resolvido

A função deste agente é ser o guardião da arquitetura do sistema "Busca Peça", prevenindo a deterioração estrutural e o acúmulo de dívida técnica. Ele aborda a questão da violação de limites arquiteturais ao analisar os Pull Requests, assegurando que as regras de negócio, interfaces de usuário e camadas de persistência e segurança (incluindo mecanismos de criptografia e controle de acesso) funcionem de maneira isolada e em conformidade com os princípios estabelecidos para o projeto.

Relação com a ODS escolhida

Ao garantir a sustentabilidade técnica do software, esse agente contribui de forma indireta para a ODS 12 (Consumo e Produção Responsáveis). Uma plataforma com uma arquitetura robusta, escalável e de fácil manutenção maximiza os recursos computacionais e humanos, estendendo a vida útil da plataforma e reforçando o modelo de economia circular do projeto.

Entradas, saídas e gatilho de acionamento

Entradas	Saídas	Gatilho de acionamento
Arquivos de código-fonte alterados, estruturas de banco de dados, definições de rotas e manifestos de infraestrutura.	Relatório de conformidade estrutural em Markdown com notificações de infrações e recomendações para a refatoração do design.	Evento (ativado por meio do webhook do repositório durante a criação ou modificação de um Pull Request).

Estilo arquitetural adotado

Microsserviço baseado em eventos. Justifica-se pela necessidade de separar a responsabilidade da análise estática e inteligência artificial do núcleo do sistema principal. Como um microsserviço autônomo, o agente pode ser dimensionado conforme a necessidade para lidar com vários Pull Requests ao mesmo tempo e pode ser atualizado sem provocar instabilidade no "Busca Peça".

Padrões de projeto aplicados (GoF e GRASP)

Categoria	Padrão adotado	Justificativa breve
Criação (GoF)	Método de Fábrica	Instancia dinamicamente o analisador apropriado (ex.: analisador de backend em Node ou de banco de dados) de acordo com o arquivo recebido.
Estrutural (GoF)	Facade	Proporciona uma interface descomplicada que esconde a complexidade da integração simultânea com analisadores de AST e APIs de LLM.
Comportamental (GoF)	Strategy	Permite alternar entre estratégias de validação arquitetural fundamentadas no domínio (ex.: regras rígidas para módulos de proteção de dados e regras flexíveis para views).
GRASP	Controlador	Recebe os eventos do repositório de código e gerencia o processo de análise estrutural, transmitindo o contexto aos especialistas em domínio.

Aplicação do padrão MVC

O agente é um serviço backend automatizado que segue os princípios do MVC. O Controller é o ponto final que aceita os webhooks de eventos de código. A lógica de validação de regras arquiteturais e a comunicação com o motor de inferência da IA são centralizadas pelo Model. A View é exclusivamente informativa, organizando e apresentando as descobertas em um modelo Markdown que pode ser utilizado na interface de Pull Request do repositório.

Diagrama de arquitetura do agente

Base tecnológica de construção do agente

O agente empregará uma estratégia híbrida que mescla uma arquitetura baseada em regras determinísticas e a orquestração de LLM por meio de prompts estruturados. Regras determinísticas, empregando parsers AST, serão implementadas para detectar quebras evidentes de padrão e acessos não autorizados entre camadas de maneira rápida e com baixo consumo de recursos computacionais. A orquestração do LLM será ativada para examinar o contexto do design, avaliar o acoplamento conceitual e elaborar sugestões humanizadas de refatoração para a equipe de engenharia.

Tecnologias e frameworks previstos

A linguagem Node.js com TypeScript, o framework web Express (para recebimento de eventos), as bibliotecas Esprima ou Babel para análise AST, e a integração por meio de API com grandes modelos de linguagem (como Google Gemini ou OpenAI) para inferência estrutural.

Modelo de interação com os demais agentes

Orquestração de múltiplos agentes. O Agente Arquiteto desempenha um papel comunicativo, oferecendo o macro-contexto e as orientações de design do módulo em avaliação para os Agentes de Validação de Código e Segurança. Isso assegura que a revisão sintática de baixo nível não aprove lógicas que, apesar de serem compiláveis, estejam alocadas na camada arquitetural errada da aplicação.

Referências utilizadas

BASS, L.; CLEMENTS, P.; KAZMAN, R. Software Architecture in Practice. 3. ed. Addison-Wesley, 2012.

GAMMA, E. et al. Padrões de Projetos: Soluções Reutilizáveis de Software Orientado a Objetos. Bookman, 2000.

LARMAN, C. Utilizando UML e Padrões: Uma Introdução à Análise e ao Projeto Orientados a Objetos e ao Desenvolvimento Iterativo. 3. ed. Bookman, 2007.

SOMMERVILLE, I. Engenharia de Software. 10. ed. Pearson, 2018.

AGENTE 3 — Agente de Documentação de Software (obrigatório)

Propósito e problema resolvido

O agente escreve e mantém a documentação técnica do projeto, o que inclui README, ADRs, docstrings e as seções de documentação formal, a partir do código-fonte e das decisões já registradas pela equipe. Ele existe porque, em equipe pequena, documentar é a primeira tarefa a cair quando o prazo aperta, e o que ninguém escreveu se perde no dia em que um integrante sai. Pior do que não ter documentação é ter documentação vencida, que engana o leitor com a autoridade de um texto oficial. A aposta aqui é sobre custo. Se documentar sai barato, deixa de ser a tarefa sacrificada. O desenvolvedor não escreve do zero, corrige um rascunho que o agente já extraiu do repositório.

Relação com a ODS escolhida

A equipe escolheu a ODS 4, Educação de Qualidade, e dentro dela a Meta 4.4: “Até 2030, aumentar substancialmente o número de jovens e adultos que tenham habilidades relevantes, inclusive competências técnicas e profissionais, para emprego, trabalho decente e empreendedorismo”. A redação brasileira adaptada troca “habilidades relevantes” por “competências necessárias, sobretudo técnicas e profissionais”. A ligação com a Escola de TI é direta. Formar alguém tecnicamente competente exige material que explique por que um sistema foi construído de determinada forma, e não apenas o que ele faz.

Para que essa relação não fique no discurso, duas decisões de arquitetura do agente saíram diretamente da ODS. A primeira é a renderização em dois níveis. O agente guarda duas Views sobre o mesmo Model, uma técnica, para quem mantém o sistema, e outra didática, para quem está aprendendo, com glossário e pré-requisitos explicitados. É por isso que o renderizador é um componente separado e parametrizável. Sem a ODS 4, uma View só resolveria.

A segunda é a captura obrigatória do rationale. Na etapa de Entrevista, o agente exige o registro da alternativa que foi descartada e do motivo do descarte. Documentação que guarda o raciocínio serve para aprender; documentação que só descreve o comportamento repete o que o código já diz melhor. O campo é obrigatório no Model, e o validador barra o documento sem ele.

O agente também toca a Meta 4.3, sobre acesso à educação técnica de qualidade. A varredura da fase de Coleta prioriza justamente o que um iniciante encontra primeiro: instalação, primeiro build e pontos de entrada da aplicação.

Entradas, saídas e gatilho de acionamento

Entradas	Saídas	Gatilho de acionamento
Código-fonte do repositório; ADRs e SPEC.md já existentes; relatório do Agente de Code Review; respostas do desenvolvedor na entrevista.	README.md; ADRs; docstrings; seção de documentação em .md e .pdf, nas versões técnica e didática; diagrama de arquitetura; relatório de lacunas.	Sob demanda, pela invocação da skill, e por evento, com hook de pre-commit previsto para a segunda iteração.

Estilo arquitetural adotado

Pipe-and-filter sobre uma organização em camadas. O agente é um pipeline de cinco filtros: Coleta, Entrevista, Síntese, Renderização e Validação. Cada estágio recebe um artefato, transforma e passa

adiante. A escolha vem da natureza do problema, porque documentar já é uma cadeia de transformações de representação. O código vira inventário estruturado, que vira conhecimento com rationale, que vira texto, que vira texto validado. Como cada filtro tem uma responsabilidade só e uma interface de dados definida, dá para trocar um deles sem tocar nos outros. Trocar o renderizador de Markdown para LaTeX não mexe na coleta nem na entrevista.

O pipeline se apoia em três camadas, com dependência em uma direção só. A camada de Instrução é o SKILL.md, que orquestra o pipeline e conversa com o usuário. A camada de Conhecimento reúne references/templates.md, checklist.md e abnt.md, onde ficam as regras de formatação, os modelos e os critérios de validação. A camada de Execução tem scripts/scan_repo.py e scripts/validate_doc.py, responsáveis pelas operações determinísticas sobre o sistema de arquivos. A Instrução conhece as outras duas; nenhuma camada de baixo conhece a de cima.

Duas alternativas foram descartadas. A arquitetura orientada a eventos pura amarraria o agente ao ciclo de vida do IDE e dificultaria rodá-lo sozinho em integração contínua; ela volta como componente de fronteira na segunda iteração, junto com o hook de pre-commit. O microsserviço exigiria infraestrutura de implantação grande demais para um agente que só mexe em arquivos locais.

Padrões de projeto aplicados (GoF e GRASP)

Categoria	Padrão adotado	Justificativa breve
Criação (GoF)	Builder	O documento sai por partes: propósito, entradas e saídas, decisões, diagrama, referências. O roteiro do SKILL.md faz o papel de Director e fixa a ordem, enquanto o mesmo processo de construção gera representações diferentes conforme o template escolhido (README, ADR, versão técnica, versão didática).
Estrutural (GoF)	Facade	O SKILL.md é a porta única do subsistema de templates, checklists, varredura e validação. Quem pede “documente este módulo” aciona tudo isso sem saber quais são as partes nem em que ordem rodam.
Comportamental (GoF)	Template Method	O esqueleto não muda. Os cinco filtros e os dois portões de aprovação humana rodam sempre na mesma ordem. Só os passos de extração e renderização são especializados por artefato.
Comportamental (GoF)	Chain of Responsibility	Os validadores rodam em cadeia: formato, completude dos campos obrigatórios e conformidade ABNT das referências. Cada elo resolve o que é seu e repassa o resto.
GRASP	Controller	O SKILL.md recebe o pedido, coordena os estágios e delega, sem guardar regra de domínio.
GRASP	Pure Fabrication	scan_repo.py e validate_doc.py não correspondem a nada do domínio “documentação”. Foram inventados para concentrar num componente determinístico as tarefas que não aceitam variação.
GRASP	Information Expert	Formatar referência em ABNT é responsabilidade de abnt.md. Definir o que torna um documento completo é responsabilidade de checklist.md.

Aplicação do padrão MVC

A aplicação é parcial, e de propósito. O Model é o conhecimento estruturado sobre o sistema documentado, ou seja, o inventário da varredura mais as respostas da entrevista, com o campo obrigatório de rationale. Nele não entra nenhuma decisão de apresentação. As Views são os templates de

renderização em references/templates.md, que transformam o mesmo Model em versão técnica, versão didática, README, ADR ou seção formal em .pdf. O Controller é o SKILL.md junto com o runtime do IDE hospedeiro, que lê o pedido, conduz a entrevista, escolhe a View e coordena os portões de aprovação.

Cabe uma ressalva. O agente não tem interface gráfica própria. A View aparece no arquivo gerado e na conversa, e o Controller é dividido com o host, já que parte do tratamento da entrada acontece no runtime do IDE, fora do alcance do agente. Preferimos registrar essa fronteira a forçar um MVC canônico.

Diagrama de arquitetura do agente

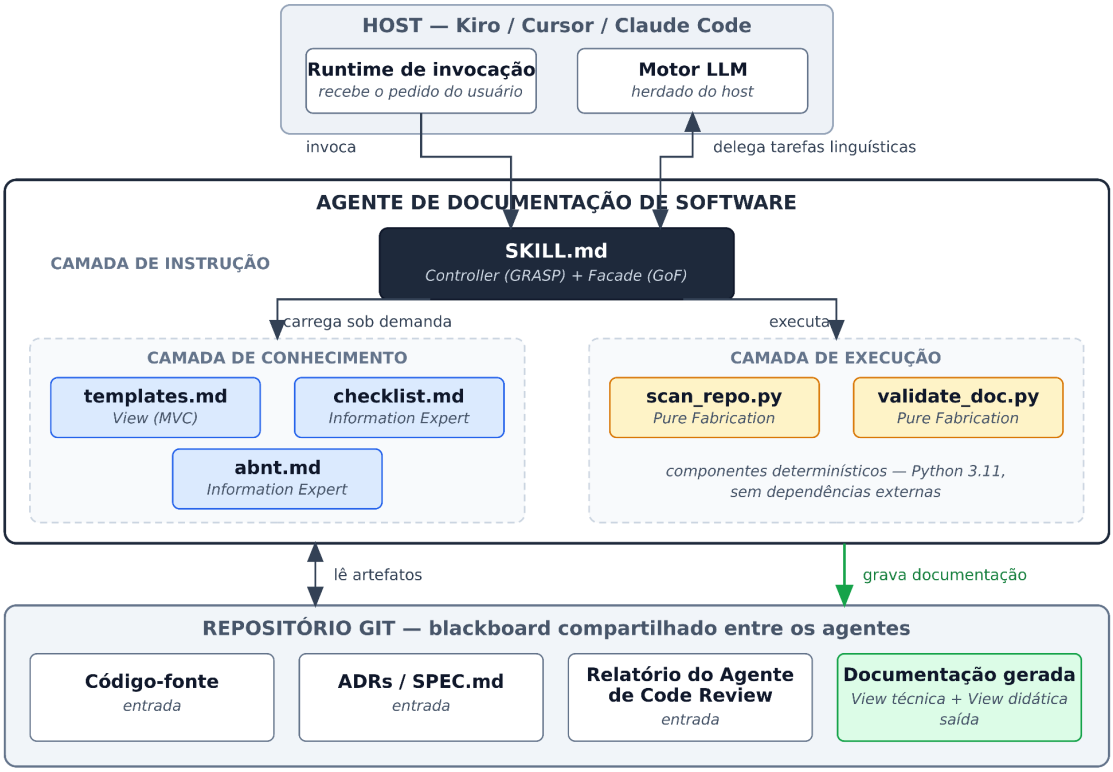


Figura 1 — Diagrama de componentes: camadas do agente, host e repositório compartilhado.

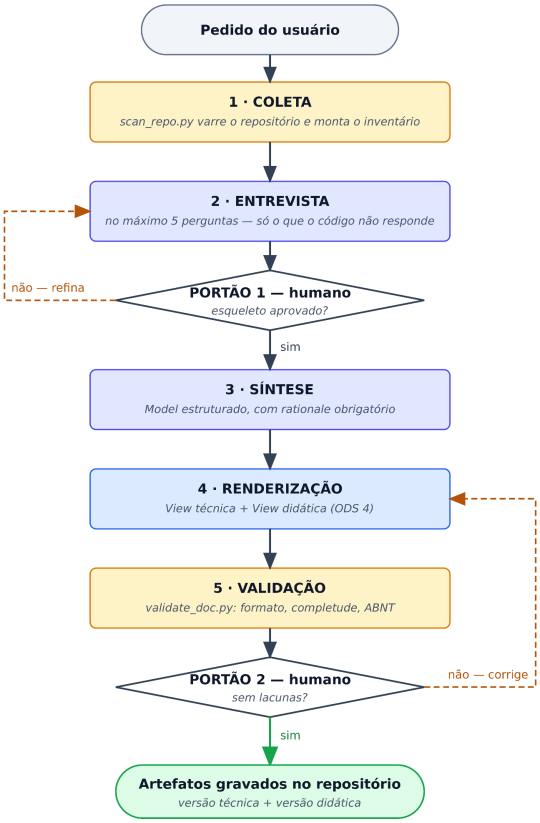


Figura 2 — Pipeline pipe-and-filter: os cinco filtros e os dois portões de aprovação humana.

Base tecnológica de construção do agente

A construção é híbrida, com a fronteira entre os dois regimes declarada. O modelo de linguagem de grande porte vem do IDE hospedeiro (Kiro, Cursor ou Claude Code) e é acionado por prompt estruturado no formato aberto SKILL.md. Ele cuida do que é linguagem: entender a intenção do código, sintetizar o propósito, conduzir a entrevista e redigir nos dois níveis de leitura.

A arquitetura própria são scripts Python determinísticos. Eles cuidam do que não pode variar entre execuções: varrer o sistema de arquivos, montar o inventário, conferir campos obrigatórios e validar formato. A divisão é uma decisão de risco. O não-determinismo do modelo custa caro, então ele foi aceito onde rende (redação, síntese, interpretação) e cortado onde erra (contagem, listagem, verificação de completude).

Escolher o formato SKILL.md em vez de um serviço próprio com API de inferência tira do caminho o custo de infraestrutura e a dependência de um fornecedor. O mesmo diretório é lido por Kiro, Cursor e Claude Code sem adaptação, e trocar o modelo por baixo não exige mexer no agente.

Tecnologias e frameworks previstos

O agente é distribuído como SKILL.md, o padrão aberto de Agent Skills, com YAML frontmatter e carregamento progressivo dos arquivos de referência. Os hospedeiros previstos são Kiro (.kiro/skills/), Cursor (.cursor/skills/) e Claude Code (.claude/skills/). Os componentes determinísticos são escritos em Python 3.11 usando só a biblioteca padrão (pathlib, ast, re, json). O Model trafega em JSON. A renderização sai em Markdown, com diagramas em Mermaid e conversão para .pdf via Pandoc. O versionamento é Git, que aqui também serve de canal entre os agentes.

As ferramentas que o agente usa têm limites declarados. Ele lê arquivos e busca por padrão sem pedir permissão, e roda no shell apenas scan_repo.py e validate_doc.py. Escrever e editar arquivo exige confirmação antes de sobrescrever documentação que já existe. Acesso à rede fica bloqueado, porque o

agente não precisa dele.

Modelo de interação com os demais agentes

Os agentes não se chamam. A conversa passa pelo repositório Git, que serve de blackboard. O Agente de Code Review deposita ali o relatório, o Agente de Arquitetura mantém os ADRs, e o Agente de Documentação lê esses artefatos na fase de Coleta e devolve documentação que os outros também consomem. É orquestração indireta por artefatos versionados.

O ganho principal é de acoplamento. Cada agente roda e é testado sozinho, sem depender de os outros estarem de pé. Há também um ganho de auditoria, já que todo o tráfego entre eles fica no histórico de commits. A alternativa seria orquestração síncrona com um coordenador central. Foi descartada por criar um ponto único de falha e por exigir mensageria desproporcional ao tamanho do projeto.

Referências utilizadas

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

LARMAN, Craig. Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.

RICHARDS, Mark; FORD, Neal. Fundamentals of software architecture: an engineering approach. Sebastopol: O'Reilly Media, 2020.

INSTITUTO DE PESQUISA ECONÔMICA APLICADA. ODS 4 — Educação de qualidade. Brasília: Ipea, [s.d.]. Disponível em: https://www.ipea.gov.br/ods/ods4.html. Acesso em: 5 set. 2026.

GRUPO DE TRABALHO DA SOCIEDADE CIVIL PARA A AGENDA 2030. Objetivo 4: assegurar a educação inclusiva e equitativa e de qualidade, e promover oportunidades de aprendizagem ao longo da vida para todas e todos. Disponível em: https://gtagenda2030.org.br/ods/ods4/. Acesso em: 5 set. 2026.

KIRO. Agent skills. Documentação oficial. Disponível em: https://kiro.dev/docs/skills/. Acesso em: 5 set. 2026.

KIRO. Steering. Documentação oficial. Disponível em: https://kiro.dev/docs/steering/. Acesso em: 5 set. 2026.

CURSOR. Agent skills. Documentação oficial. Disponível em: https://cursor.com/docs/skills. Acesso em: 5 set. 2026.